

CAMPUS PULSE

Student Activity Analyzer

Python Mini Project Report

Submitted by	ANMOL RATHORE _____
Registration No.	26BCE11629 _____
Program	B.Tech - Computer Science and Engineering
Institution	VIT Bhopal University
Academic Year	2026-27
Submission Date	29 September 2026

A simple console-based system for recording study habits,
viewing progress, and getting practical study feedback.

Contents

1. Introduction
2. Problem Statement
3. Functional Requirements
4. Non-functional Requirements
5. System Architecture
6. Design Diagrams
7. Design Decisions & Rationale
8. Implementation Details
9. Screenshots / Results
10. Testing Approach
11. Challenges Faced
12. Learnings & Key Takeaways
13. Future Enhancements
14. References

Report note

The current implementation keeps data in memory only. No SQL or other external database is used, so an ER diagram is not applicable in this version.

1. Introduction

Campus Pulse is a small Python application designed to help a student keep track of everyday study habits in one place. The program uses a simple console interface, so it can be run directly from a Python terminal without installing a database or a separate graphical interface.

The project focuses on a practical problem: students often know that they studied, completed assignments, or prepared for an exam, but they do not have an easy way to look back at those habits. Campus Pulse records a few useful measures - study hours, assignments, workload, exam confidence, sleep hours, and the subject studied - and turns them into a small dashboard and simple suggestions.

The current version is intentionally straightforward. Lists, dictionaries, functions, loops, conditional statements, and basic calculations are enough to build the full application. This keeps the project easy to understand for a first-year programming course while still showing a complete flow from input to analysis.

What the project does

It records daily study information, calculates averages, identifies the most-studied subject, gives a simple pulse status, shows priorities, and offers a Quick Study Check-in.

Project objectives

- Create a simple menu-driven Python application that a student can use without extra software.
- Practice core Python concepts through a real, usable problem.
- Turn raw daily inputs into understandable summaries instead of displaying only stored data.
- Validate user input and handle empty-data cases gracefully.
- Keep the code modular so individual features can be read and tested separately.

2. Problem Statement

Students manage several small academic tasks every day: studying different subjects, completing assignments, preparing for exams, and maintaining a healthy routine. When this information is not tracked, it becomes difficult to notice simple patterns such as consistently low study time, high workload, or low confidence before an exam.

The problem is therefore to build a lightweight activity analyzer that accepts a small set of daily inputs, stores them temporarily during program execution, summarizes the information, and gives practical feedback. The solution should remain simple enough to demonstrate introductory Python concepts and should not depend on a database.

Problem area	Need addressed by Campus Pulse
Study tracking	Records study hours for each daily entry.
Assignment load	Keeps a count of assignments so workload can be viewed in context.
Exam confidence	Stores a 1-10 confidence rating and uses it in the pulse and priority logic.
Routine balance	Records sleep hours and mentions rest when average sleep is low.
Subject-wise view	Adds total study hours and entry counts by subject.
Actionable feedback	Turns the numbers into priorities and a Quick Study Check-in.

3. Functional Requirements

The following requirements describe what the program must do in its current form.

ID	Requirement	Description
FR-01	Add Daily Entry	Accept student name, subject, study hours, assignments, workload, confidence, and sleep hours; then store the entry.
FR-02	Input Validation	Reject blank names/subjects and invalid numeric ranges or non-numeric values.
FR-03	View Dashboard	Display entry count, averages, most-studied subject, pulse status, and simple level bars.
FR-04	View Study History	Display all entries with their recorded fields.
FR-05	Today's Priority	Create study priorities from low confidence, low study time, high workload, and low sleep.
FR-06	Subject Analysis	Show total study hours and number of entries for each subject.
FR-07	Delete Last Entry	Allow the latest entry to be removed after confirmation.
FR-08	Quick Study Check-in	Give a short natural-language interpretation of study time, confidence, workload, and sleep, plus one suggestion.
FR-09	Exit Safely	Close the application when the user selects option 8.

4. Non-functional Requirements

Quality	Requirement
Usability	Menu labels and messages should be readable and understandable to a student using a terminal.
Reliability	Invalid input should not crash the program; the user should be asked to correct it.
Maintainability	Functions should have one clear purpose so features can be edited independently.
Portability	The program should run on a standard Python installation with no external packages.
Performance	For the intended small number of entries, calculations should complete immediately.
Simplicity	The design should remain appropriate for an introductory Python project instead of adding unnecessary infrastructure.
Data handling	Current data is stored only during execution and is not sent to any external service.

5. System Architecture

Campus Pulse uses a small three-part structure. The user interacts with the console interface, the selected feature function performs the requested operation, and the shared in-memory list provides the current data. Helper functions support validation and repeated operations.

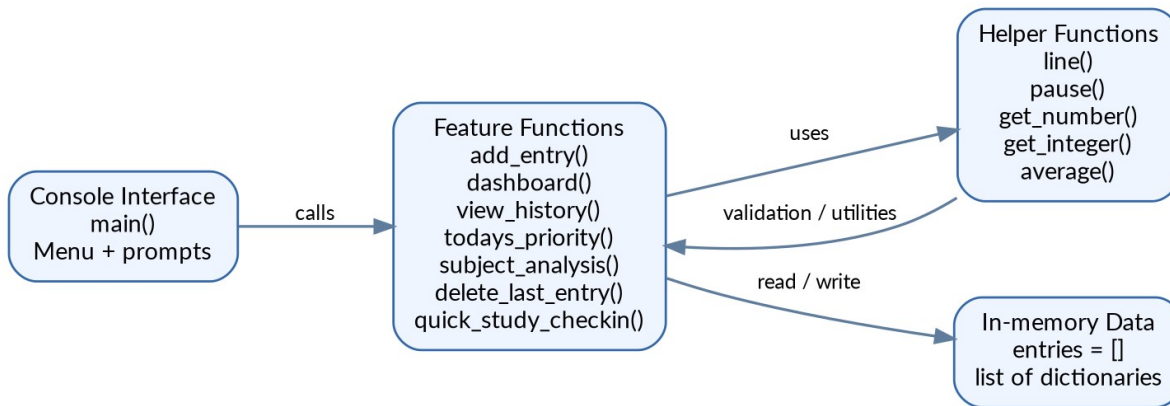


Figure 1. High-level component architecture of Campus Pulse.

Layer	Main responsibility	Examples from code
Presentation / Console	Displays menus and asks for input.	<code>main()</code> , <code>line()</code> , <code>pause()</code>
Application Logic	Processes entries and produces summaries.	<code>add_entry()</code> , <code>dashboard()</code> , <code>todays_priority()</code> , <code>quick_study_checkin()</code>
Data / State	Holds records while the program is running.	entries list containing dictionaries
Utilities	Keeps repeated validation and simple calculations in one place.	<code>get_number()</code> , <code>get_integer()</code> , <code>average()</code>

Important architecture choice

There is no persistent database layer in the current version. When the program closes, the in-memory entries list is cleared. This keeps the project small and avoids SQL setup.

6. Design Diagrams

6.1 Use Case Diagram

The main actor is the Student. The student can add and review activity data, analyze subjects, remove the most recent entry, and use the Quick Study Check-in.

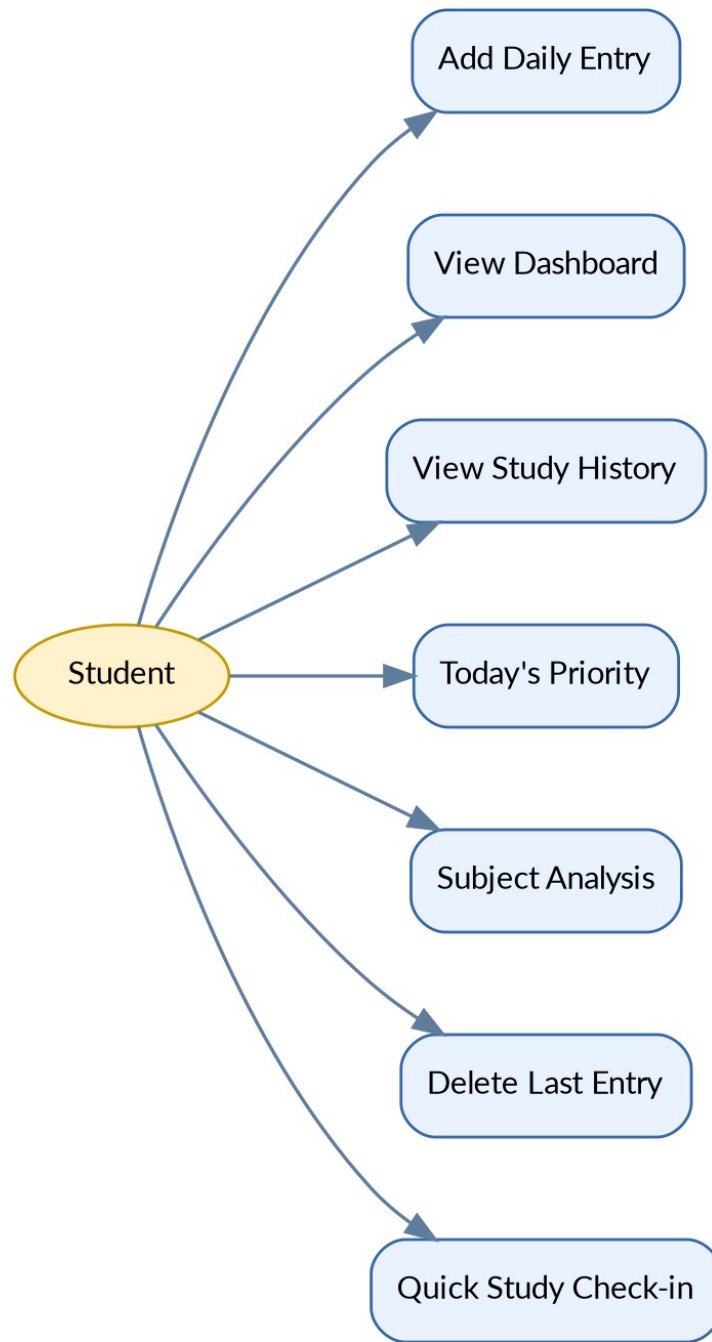


Figure 2. Use case diagram.

6.2 Workflow Diagram

The workflow starts at the menu, branches to the selected feature, validates or analyzes data, displays the result, and then returns to the menu until the user exits.

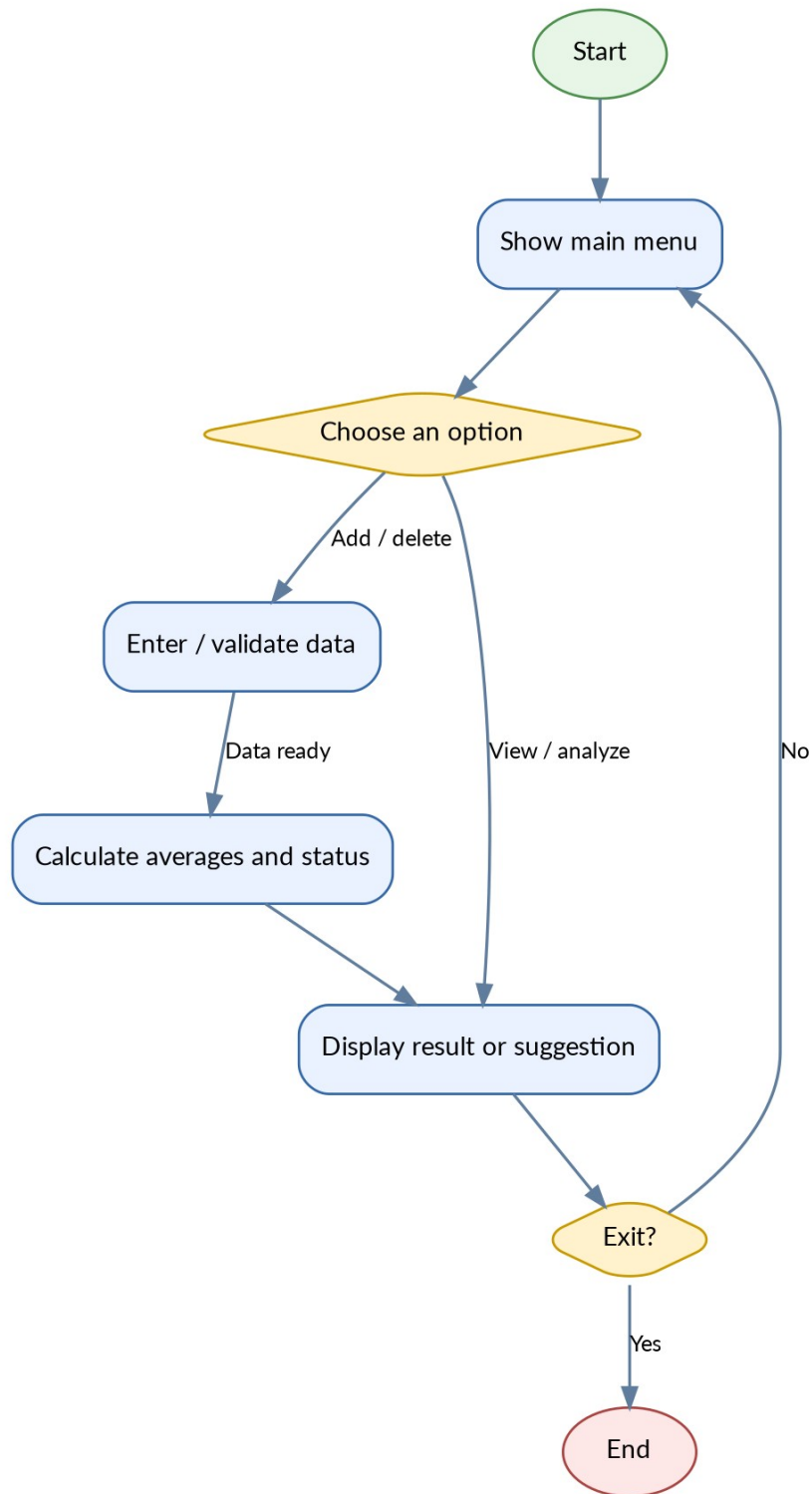


Figure 3. Overall workflow.

6.3 Sequence Diagram

The sequence below illustrates a typical Add Daily Entry interaction. Input is collected from the student, processed by the feature function, and stored in the shared entries list before a confirmation message is shown.

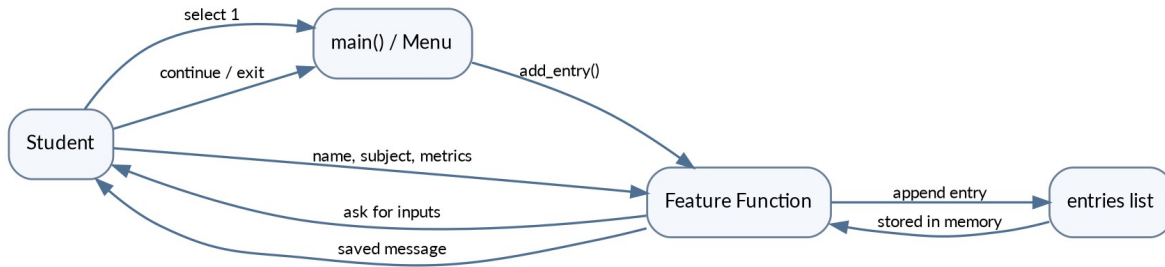


Figure 4. Sequence for adding a daily entry.

6.4 Class / Component Diagram

The project is function-based rather than class-based. Therefore, a component-style diagram is more accurate than inventing object-oriented classes that do not exist in the implementation.

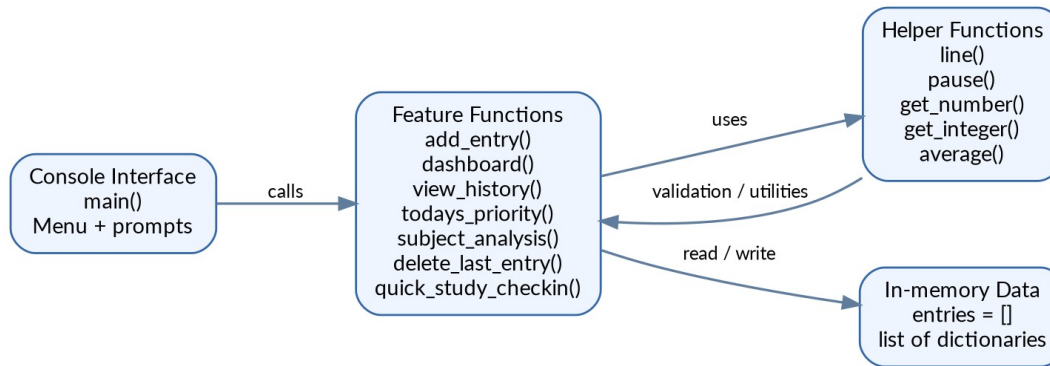


Figure 5. Function-based component view.

6.5 ER Diagram

Not applicable in the current version

The project does not use persistent storage or a relational database. Entries are stored as dictionaries inside a Python list while the program is running. Therefore, there are no tables, keys, or relationships to model in an ER diagram.

7. Design Decisions & Rationale

Decision	Rationale
Use a console interface	A terminal application is quick to build, easy to demonstrate, and matches the learning goals of an introductory Python project.
Use lists and dictionaries	A list naturally stores multiple entries, while dictionaries make fields such as subject, workload, and sleep easy to access by name.
Keep data in memory	The current scope does not require persistent storage. Avoiding SQL keeps installation and explanation simple.
Separate functionality into functions	Each menu feature has its own function, which improves readability and makes testing easier.
Validate input through helper functions	<code>get_number()</code> and <code>get_integer()</code> prevent invalid values from reaching the calculation logic.
Use a simple score for Campus Pulse	A small rule-based score gives the dashboard an understandable status without adding advanced machine learning.
Add natural-language check-in feedback	The Quick Study Check-in makes the numeric data easier for a student to understand and act on.

8. Implementation Details

The implementation is contained in a single Python file, main.py, and uses the standard datetime module. No external packages are required.

8.1 Data structure

```
entries = []

entry = {
    "date": datetime.now().strftime("%d-%m-%Y"),
    "name": name,
    "subject": subject,
    "study_hours": study_hours,
    "assignments": assignments,
    "workload": workload,
    "confidence": confidence,
    "sleep_hours": sleep_hours
}
```

Each daily record is a dictionary. All dictionaries are kept in the entries list. This makes it possible to loop through records for averages, history, subject analysis, and priority rules.

8.2 Input validation

```
while True:
    try:
        value = float(input(prompt))
        if minimum <= value <= maximum:
            return value
        print("Enter a value between", minimum, "and", maximum)
    except ValueError:
        print("Please enter a valid number.")
```

The same pattern is used for integer input. Blank name and subject values are also checked before a new record is saved.

8.3 Analysis logic

- average(key) adds the chosen field across all entries and divides by the number of records.
- most_studied_subject() groups study hours by subject and returns the subject with the highest total.
- get_pulse_status() converts average study time, workload, and confidence into a small 1-9 score and maps it to GOOD, MODERATE, or NEEDS ATTENTION.
- todays_priority() and quick_study_checkin() apply simple if/elif rules to turn the averages into readable advice.

The main menu reads the user choice, calls the matching feature function, and repeats until option 8 is selected.

9. Screenshots / Results

The following are representative console results generated from the final main.py used for this report. They show the updated menu, dashboard calculations, and the added Quick Study Check-in feature.

```
Representative console result - final version

CAMPUS PULSE
Student Activity Analyzer
=====
1. Add Daily Entry
2. View Dashboard
3. View Study History
4. Today's Priority
5. Subject Analysis
6. Delete Last Entry
7. Quick Study Check-in
8. Exit
=====
Enter your choice: 8

Thanks for using Campus Pulse!
Keep studying at your own pace. Good luck!
```

Figure 6. Final menu showing Quick Study Check-in instead of About Project.

Representative console result - dashboard and check-in

CAMPUS PULSE - DASHBOARD

=====

Total entries : 1
Average study time : 4.50 hours
Average assignments: 2.00
Average workload : 6.00 /10
Average confidence : 7.00 /10
Average sleep : 7.00 hours
Most studied : Python
Campus Pulse : MODERATE

Study Level

[#####----] Good

Confidence Level

[#####] High

Workload Level

[#####----] Moderate

QUICK STUDY CHECK-IN

=====

Study: You are getting some work done. A little more consistency could help.
Confidence: You seem comfortable with your preparation.
Workload: Your workload is manageable, but keep an eye on deadlines.
Sleep: Your average sleep is looking okay.

One simple suggestion:

Keep the routine steady and focus on regular revision.

Figure 7. Representative dashboard and Quick Study Check-in output.

Observed result

The sample run successfully saved one entry, calculated averages, identified Python as the most-studied subject, produced a MODERATE pulse status, and generated a short study suggestion.

10. Testing Approach

Testing focused on the normal menu flow, empty-data behavior, invalid menu input, entry creation, analysis features, deletion, and the new Quick Study Check-in. Each test was run against main.py, and the expected output was checked.

ID	Test	Action	Expected result	Status
TC-01	Program starts	Run main.py and select Exit	Menu appears; program exits normally	PASS
TC-02	Add entry + dashboard	Enter a complete record, then open Dashboard	Record saved; averages and pulse displayed	PASS
TC-03	Quick Study Check-in	Add a record, then choose option 7	Readable summary and suggestion displayed	PASS
TC-04	Empty dashboard	Choose Dashboard before adding data	Helpful no-data message shown	PASS
TC-05	Invalid menu	Enter 9 at the main menu	Program rejects the choice without crashing	PASS
TC-06	Delete last entry	Add a record, choose Delete, confirm y	Latest record removed	PASS
TC-07	History + subject analysis	Add a record, open both views	History and subject totals displayed	PASS

Testing result

7 out of 7 planned smoke/functional scenarios passed for the final version used in this report.

11. Challenges Faced

The main development challenges were less about complex algorithms and more about making a small program behave reliably in real use.

- Handling incorrect numeric input without allowing the program to crash required reusable validation loops.
- Keeping the dashboard calculations correct across multiple entries required a common `average()` helper.
- Showing useful output when there are no entries required explicit empty-data checks in several features.
- Subject analysis needed a way to combine hours for repeated subjects while keeping the code simple; a dictionary was used for this.
- The project menu had to stay consistent as features were added, especially after replacing the old About Project option with Quick Study Check-in.
- The final version had to remain simple enough for a first-year Python project while still offering meaningful functionality.

12. Learnings & Key Takeaways

This project helped connect individual Python concepts into one complete application. Instead of writing isolated practice programs, the code now has a clear input -> process -> output flow.

Learning	How it was used
Functions	Separate menu features and repeated utilities into small blocks.
Lists and dictionaries	Store and work with multiple student activity records.
Loops	Repeat menus, validate input, and process all stored entries.
Conditional logic	Classify study level, workload, confidence, and priorities.
Exception handling	Manage invalid numeric input using try/except.
Basic data analysis	Calculate averages and find the most-studied subject.
Program design	Organize a complete console application rather than a single exercise.
Debugging mindset	Test empty data, invalid choices, deletion, and normal workflows.

Key takeaway

A useful project does not need a large technology stack. Clear requirements, sensible data structures, validation, and readable functions can already produce a complete working application.

13. Future Enhancements

1. Persistent storage: save entries to JSON, CSV, or SQLite so data remains after the program closes.
2. Student login: allow different students to keep separate activity records.
3. Graphical interface: build the same features as a desktop or web interface for easier use.
4. Charts and trends: show weekly study hours, workload patterns, subject distribution, and confidence trends.
5. Export reports: generate a CSV or PDF summary for a selected date range.
6. Smarter reminders: remind the student about high workload, low confidence, or missing entries.
7. More detailed analytics: compare study time with confidence and workload over time.
8. Cloud or mobile version: allow access to the same data from multiple devices after authentication and secure storage are added.

These improvements can be added later without changing the core idea of the project. The current function-based structure provides a simple base for extending the application.

14. References

9. Python Software Foundation. Python 3 Documentation - Data Structures.
<https://docs.python.org/3/tutorial/datastructures.html>
10. Python Software Foundation. Python 3 Documentation - More Control Flow Tools.
<https://docs.python.org/3/tutorial/controlflow.html>
11. Python Software Foundation. datetime - Basic date and time types.
<https://docs.python.org/3/library/datetime.html>
12. Python Software Foundation. Built-in Exceptions.
<https://docs.python.org/3/library/exceptions.html>
13. Python Software Foundation. Python Standard Library. <https://docs.python.org/3/library/>

Source note

The diagrams in this report are original illustrations prepared for this project report. The screenshots/results are representative outputs generated from the final project code.